



µLab 3 • Week 2 / Day 2 • Driver's handout

Deploying Flyte and running a pipeline on it

Group _____ Date _____ Time _____

Driver (types) _____ Mapper (looks up) _____ Referee (demos) _____

STEP 1 • CHECK YOUR TWO TOOLS, THEN START THE FLYTE CLUSTER

The round needs `flytectl` and `uv`, both installed this morning. Check them first. If either line prints nothing, install it now from the 08h45 slide; it takes seconds, and everything below depends on it.

```
flytectl version
uv --version
```

Now free the port. The Spark cluster and the Flyte one both publish **6443**, and two programs cannot hold one port. Nothing is lost: `docker start k8s` brings Spark back later with its data.

```
docker rm -f ui
docker stop k8s
flytectl demo start
```

<code>docker rm -f ui</code>	Removes the port-forward container from µLab 2. Nothing needs it now.
<code>docker stop k8s</code>	Stops the container running the Spark cluster and releases port 6443. It does not delete it.
<code>flytectl demo start</code>	Creates a container named <code>flyte-sandbox</code> holding a Kubernetes cluster with Flyte installed on it. It publishes 6443 for the cluster and 30080 for the web console.

If it asks `delete existing sandbox cluster [y/n]`, answer **n**. That question appears only when you already have a sandbox, and **n** keeps it: it replies `Existing details of your sandbox` and carries on.

The last thing it prints is one line. **Run it**, in the terminal you will use for step 2. Your own home directory appears where `$HOME` is here:

```
export FLYTECTL_CONFIG=$HOME/.flyte/config-sandbox.yaml
```

Without it, step 2 does not know where your cluster is.

Time it took to start _____

With the image already on your laptop this is about two minutes. Much longer means it is still pulling, and panel 7 says what to do.

STEP 2 • RUN THE TWO-TASK WORKFLOW ON THAT CLUSTER

`hello.py` came in with this morning's `git pull`, so run this from inside `lab`. It has two tasks and one workflow: `split` turns a sentence into a list of words, `tally` counts them. The point is not the count. It is that the list of words leaves one task and arrives in another, and that Flyte checked the two types agreed before anything ran.

```
cd lab
```

```
uv run --python 3.12 --with flytekit==1.16.28 \
  pyflyte run --remote hello.py count_words --sentence "the quick brown fox jumps"
```

`uv run --python 3.12` Runs the command in a throwaway environment on Python 3.12. `flytekit` refuses to install on 3.9 or on 3.13 and later, and your laptop has whichever it has; this sidesteps the question entirely.

`--with flytekit==1.16.28` The Flyte SDK, pinned. Flyte 2 exists and is different; everything here is the 1.x line.

`pyflyte run --remote` Sends the file to the cluster and runs it **there**. Without `--remote` it would run in this terminal and no cluster would be involved.

`count_words` Which of the things in the file to run. A file may hold several workflows.

--sentence "...". An input of the workflow. `pyflyte` built this flag from the function's signature, which is why a typed signature is worth having.

It returns in about four seconds and prints a URL. Those seconds are the **submission**, not the run: the run is only starting.

The URL it printed ends in an execution id. Write that id here: _____

STEP 3 · FIND THE RUN IN THE CONSOLE

Open `http://localhost:30080/console` and find your execution. Wait until it says **SUCCEEDED**. Expect tens of seconds, nearly all of it spent starting a container for each task.

Write down, from the console:

How many tasks ran _____ How long the whole execution took _____

What value `tally` received from `split` _____

Then answer this, in one sentence. A shell script running the same two steps would have printed the same number. Name one thing the console is showing you that the shell script could not have told you.

*Solved means panels 1 to 3 are done: the console shows one execution of `count_words` in phase **SUCCEEDED**, with two tasks. The referee ticks the bar below at that moment, not at the caucus.*

Reached this line? Then you have solved the challenge.



Referee: tick the box on the left now, and write the time. Do not wait for the caucus. Then carry on below — the extensions are not optional, they are the rest of the round.

Solved at _____ Referee's initials _____

4 · SOME DEFINITIONS

These are the words the console puts on screen. They are here, below the bar, because none of them is needed to finish the core.

task	A Python function marked <code>@task</code> . It runs in its own container, on its own, and its arguments and return type are declared. Flyte will not connect two tasks whose types disagree.
workflow	A Python function marked <code>@workflow</code> that calls tasks. It does not run them: calling a task inside a workflow records that the output of one becomes the input of another. The graph is built from those recordings, then executed elsewhere.
execution	One run of a workflow, with an id such as <code>an4w2r8ql4pw6sf9tkw9</code> . Every execution is kept, with its inputs, its outputs and its logs. This is the thing a shell script does not have.
launch plan	A workflow plus a set of input values, saved under a name. The console labels your run <code>LAUNCH_PLAN</code> because <code>pyflyte run</code> made one for you.
sandbox	The single container <code>flytectl demo start</code> creates. It holds a Kubernetes cluster, the Flyte services, a database, an object store and an image registry. It is the laptop deployment of Flyte, not the production one, and everything you do here would be done the same way against a real cluster.

5 • EXTENSION • MAKE THE SECOND RUN COST NOTHING

What this shows, before you do it. Both tasks ran in containers and took about 37 s. Nothing about `split` depends on anything but its input. If the input has not changed, running it again is waste. Flyte can be told that.

Copy `hello.py` to `hello_cached.py` and change **both** `@task` lines to:

```
@task(cache=True, cache_version="1.0")
```

`cache_version` Your label for “the code inside this task”. Change the body of the task and you must change this string, or Flyte will hand back the old answer.

```
uv run --python 3.12 --with flytekit==1.16.28 \
  pyflyte run --remote hello_cached.py count_words --sentence "the quick brown fox jumps"
```

Run it, **wait for it to reach SUCCEEDED**, then run the identical command again. Waiting matters: a task’s result only enters the cache when it finishes, so two runs fired back to back both miss.

First run, elapsed _____ Second run, elapsed _____

Then open the second run in the console and look at its two tasks. Write down what the console says about them that it did not say about the first run.

Observed on 2026-09-24: 39.1 s, then 0.085 s, and zero pods. The second run did not do the work faster. It did not do it at all: Flyte handed back the result it had already stored.

6 • EXTRA MILE • BREAK THE TYPES ON PURPOSE

Change `tally` to take a `str` instead of a `List[str]`, leaving everything else alone, and run it again. Write down **when** the failure happens: before the cluster is contacted, after submission but before a task starts, or inside a running task.

Then say which of those three a shell script chaining two commands would have given you.

7 • ANNEX • GETTING PAST THREE THINGS THAT GO WRONG

Bind for 0.0.0.0:6443 failed: port is already allocated

connection refused on port 30080

flytekit **will not install**

The image never arrived, and the sandbox will not start

The Spark cluster is still running. `docker stop k8s`, then start the sandbox again. This is panel 1 and it is the most common failure of the round. The sandbox is up but Flyte inside it is not ready yet. `flytectl demo status` reports whether it is. A warm start took 110 s on 2026-09-23.

Your Python is older than 3.10 or newer than 3.12. That is what `uv run --python 3.12` in panel 2 is for; do not drop it.

Take the image off the USB stick: `gunzip -c flyte-sandbox.tar.gz | docker load`, then start again. If even that is not an option, drop `--remote` from the step 2 command: `pyflyte run hello.py count_words --sentence "..."` runs the same workflow in this terminal, with no cluster and no image. You lose the console, the execution history and panel 5, and you keep the part where the types are checked and the output of one task becomes the input of the next.

Leave the sandbox installed when you go. Monday’s session starts from it. Tearing it down costs the 700 MB download again to get back.